



## Lesson 8: Final Project — Our Own Mini Game! 🎮

### Big Idea

Now that we know variables, input, decisions, loops, and lists, we have everything we need to design and build our very own mini game from scratch!

### Learning Objectives

- Plan a simple game on paper before coding it (inputs, rules, win/lose conditions).
- Combine variables, `input()`, `if / elif / else`, loops, and lists into one original program.
- Use the `random` module to add unpredictability to a game.
- Test their own program, find at least one bug, and fix it independently.

### Teacher Preparation

- Have Thonny ready.
- Print or display a simple "Game Planning Sheet" (see Activities.md) for students to fill out before coding.
- Have the Word Guessing Game (Project 06) available as a model example if students want inspiration, but encourage original ideas too.

### Required Files

- `Student_Lesson_Guide.md`
- `Activities.md`
- `Challenges.md`
- Thonny IDE or Repl.it

### Teaching Flow (approx. 55–60 minutes, can span 2 sessions)

1. Warm-up / Hook — 5 min
2. Concept Introduction — 5 min

3. Teacher Demonstration — 10 min
4. Guided Coding — 10 min
5. Mini Challenge — 5 min
6. Main Activity (build own game) — 20 min
7. Check for Understanding + Recap — 5 min

## Warm-up / Hook (5 min)

Ask: "If you could design ANY simple game using what we've learned — guessing, quizzes, or random choices — what would you build?" Let a few students share ideas out loud.

 **Coding Mission:** Today you become a real game designer AND a real game programmer — you'll plan it, build it, test it, and fix it!

## Concept Introduction

- "Every game needs: something for the player to do (input), rules for what happens (if/else), and often, a way to keep playing (loops) or randomness (lists + random)."
- "We're going to plan our game FIRST on paper, just like real game designers do, before writing any code."
-  **Think Like a Coder:** "Good programmers plan before they code — it saves time and helps avoid confusion later!"

## Teacher Demonstration

Model planning and building a simple example: a "Lucky Number Game." 1. Plan on the board: Player guesses a number 1-5. Computer picks a random number 1-5 using `random.randint(1, 5)`. If they match, player wins. 2.

Type and run: `python import random`

```
secret_number = random.randint(1, 5) guess = int(input("Guess a number between 1 and 5: "))
```

```
if guess == secret_number: print(f"🎉 You guessed it! You win!") else: print(f"Not quite! The number was {secret_number}. Try again next time!") `` 3. Run it a few times, showing that sometimes they win, sometimes they lose, because of randomness.
```

## Guided Coding

Have students type and run the same Lucky Number Game, then discuss as a class what other rules or features could be added (attempts, hints, replay loop).

## Mini Challenge 🎮

Ask students to add a replay loop using `while` so the player can guess again:

```
import random

play_again = "yes"
while play_again == "yes":
    secret_number = random.randint(1, 5)
    guess = int(input("Guess a number between 1 and 5: "))
    if guess == secret_number:
        print("🎉 You guessed it! You win!")
    else:
        print(f"Not quite! The number was {secret_number}.")
    play_again = input("Play again? (yes/no) ")

print("Thanks for playing!")
```

## Main Activity / Project

Students plan and build their OWN original mini game using the Game Planning Sheet in [Activities.md](#), then code it in Thonny using any combination of variables, input, if/else, loops, and lists they choose. Encourage students to test it at least 3 times and fix any bugs they find.

## Fun Challenge (optional, harder) ★

Add a scoring or attempts system to their game (e.g., limited guesses, or points for winning quickly), similar to Project 06 (Word Guessing Game).

## Check for Understanding

- "What are the main ingredients every simple game needs?"
- "Why is it helpful to plan a game before coding it?"
- "What tool do we use to add randomness to a game?"
- "What did you do when you found a bug in your own game?"

## Common Student Mistakes

- **Trying to code without planning first:** leads to confusion about what the game should do. Fix: always fill out the planning sheet first.

- **Forgetting `import random` when using randomness.** Fix: add it at the very top of the file.
- **Loop that never ends because the replay variable never gets a new value inside the loop.** Fix: make sure `play_again = input(...)` is inside the loop so it updates each time.
- **Mixing text and number comparisons** (e.g., comparing a guess to a number without `int()` conversion). Fix: convert `input()` to `int()` whenever comparing to a number.
- **Overcomplicating the first version:** trying to add too many features at once. Fix: build the simplest working version first, then add features one at a time.

## Differentiation

- **Struggling support:** Offer the Lucky Number Game as their final project with light personalization (change the range, change the messages).
- **Standard:** Design and build an original simple game using the planning sheet.
- **Extension for advanced learners:** Add scoring, limited attempts, replay loops, and multiple rounds; combine ideas from Projects 04-06.

## Recap

"Today you used everything from this whole course — variables, input, decisions, loops, and lists — to build your own original mini game! That is a huge accomplishment. You are officially Python game builders!"

## Take-Home / Creative Challenge

Ask students to demo their finished mini game to a family member and get feedback on what they liked and what could be added next.

## Teacher Talking Points

- "Look how far you've come — from typing your very first `print('Hello, world!')` to building your OWN original game, all by yourself!"
- "Real game developers plan first, then build, then test, then fix bugs — you just did exactly that, like a professional!"
- "It's totally normal for your first version to have bugs — that's not failure, that's just part of coding. Finding and fixing bugs IS coding!"
- "You should be really proud of yourself. You built something real, from nothing, using just words and your own ideas."



## Victory Moment

Celebrate every student sharing their finished mini game with the class — this is the capstone moment of the whole course, and every student's game is worth celebrating!